# Are LLM Written Unit Tests Flaky?

Progress Report 2, ECE 382V Software Testing in the Era of Nondeterminism, Fall 2026

Shaunak Kapur (sk62288)
The University of Texas at Austin
Austin, Texas, USA

Manas Pathak (map7848)
The University of Texas at Austin
Austin, Texas, USA

## 1 SUMMARY

The main problem after Progress Report 1 was that we had only 3 flaky LLM tests to study. We changed the plan to attack that shortage directly.

(1) **Known flaky code (E2).** We took projects where Gruber et al. [2] found a flaky test, reproduced that test on our own runner, and asked four LLMs to write tests for the same code without seeing any test file. E2 now covers 41 projects: 17 from their hand labeled sample and 24 from an extension batch that we chose from their unlabeled data by a fixed rule (Section 4.2).
(2) **More runs (E1).** We reran all 96 suites of the 12 library study, 600 times per generated test and 200 times per human test, now with two OpenAI models, GPT 5.4 mini and GPT 4.1 mini (Section 4.1).
(3) **Why (E3).** For the flaky E2 tests we reran each one alone and searched for the test that pollutes it, as iDFlakies [4] does (Section 4.3).
(4) **Mitigation.** We tried two fixtures, three prompt versions, and longer rerun filters (Section 4.5).

A test is flaky, or *flips*, if it both passed and failed on the same machine. We now have 16 distinct causes of flaky LLM tests in 15 projects (74 tests a developer would keep, in 29 suites), plus 17 tests that a one run filter would remove (Table 1). Each has a root cause label. Three preliminary results stand out.

- **Each model's flaky tests share one main cause** (Table 11), though each pattern comes from few suites, and Claude and OpenAI suites came from different pipelines (Section 3). GPT 4.1 mini leaves randomness unseeded (12 tests in 7 suites). GPT 5.4 mini is the only model that reloads modules, and 24 of its tests depend on test order because of it, all from 3 suites in the extension batch. In E2, Claude Haiku and Claude Sonnet flip only where the code under test is itself nondeterministic, through Python's per process salted string `hash()` in cuckoopy or an unstable object hash in kevinarpe-rambutan3.
- **Sonnet had the fewest flaky tests, but no model difference is significant yet.** Over 41 projects, GPT 4.1 mini has a kept flaky test in 6 projects where Sonnet has none, and the reverse never happens (exact sign test, $p = 0.031$, above the 0.0083 threshold for six model pairs). Neither batch is significant alone (Table 9). More suites per project is our first next step (Section 7).
- **A fixture removes the two causes it can reach, on the tests it was built for.** Seeding every generator inside the test call (fixture M1) removed all 12 unseeded randomness tests. Adding a restore of interpreter state after each test (fixture M2) removed all 24 order dependent ones, and GPT 5.4 mini went from 25 kept flaky tests to 1. We designed both fixtures after seeing these tests, so the new suites of Section 7 are the held out check.

**Table 1: Flaky LLM tests we can now study. Kept means a one run generate and filter step would keep the test. Removed means the filter would drop it, or would drop the only test it flips after (2 tests). The next columns count the test functions, suites, and projects the kept tests come from, and Causes counts distinct pairs of project and root cause, our measure of independent failures. A project can appear in two rows, so totals count distinct ones. Guard prompts are defined in Section 4.5.**

| Source | Kept | Removed | Functions | Suites | Projects | Causes |
|---|---|---|---|---|---|---|
| E1, large rerun campaign | 15 | n/a | 8 | 5 | 3 | 4 |
| E2 plain, first 17 projects | 13 | 12 | 13 | 7 | 5 | 5 |
| E2 plain, extension (24) | 36 | 2 | 30 | 12 | 6 | 6 |
| E2 guard prompts v1, v3 | 10 | 3 | 8 | 5 | 3 | 3 |
| Total | 74 | 17 | 59 | 29 | 15 | 16 |

The 74 kept tests come from only 29 suites and 16 causes, so we compare models per project, and all 16 order dependent jmcm tests count as one cause. Two of the 3 tests from Progress Report 1 flipped again and are inside the 15 of E1. E1 does not study removed tests, so that column is empty. The 2 removed tests that pass their first run flip only after a test that the filter rejects (Section 4.3), so they would never flip in a filtered suite. The fixture runs of Section 4.5 rerun these same tests and found one more flaky test, a rare docci clock test, which we leave out of this table.

## 2 RESPONSE TO THE PROGRESS REPORT 1 FEEDBACK

Table 2 answers each comment.

## 3 STUDY DESIGN

Table 3 defines every condition. The *feature prompt* names the time, randomness, thread, and file features of each library so the model tests them. The *TestPilot style* prompt shows only signatures, docstrings, and README examples, following the TestPilot generator [9]. The *plain* prompt only asks for pytest unit tests for the given source. Every prompt is in `prompts/README.md`, word for word.

**Table 2: Each comment on Progress Report 1 and what we changed**

| Comment | What we did |
|---|---|
| The subject filter numbers were packed into one sentence. What does "and we say so" mean? | The funnel is now a table (Table 4). The phrase meant that we oversample the first stratum on purpose. We now say that directly and give the reason below the table. |
| Why randomly sample, and why not run more? | The sample only estimated the pass rate of the last funnel rule and picked no subject, and E2 replaced the plan to add funnel subjects. We followed your suggestion and run more: 600 runs per generated test in E1 instead of 70 to 90, and 200 per test on 41 projects known to have flaky tests (E2). |
| Why not use Python flaky tests that prior datasets already found? | We now do. We reproduced 51 flaky tests of Gruber et al. and used 41 of their projects (Table 6). |
| Can a study rest on three flaky tests? | No, so we built E2 to find more. We now have 74 kept flaky LLM tests and 17 that a filter removes (Table 1). They cluster in a few suites, so we compare models per suite and per project. |
| Are LLMs just good at avoiding flakiness, and why? | Some are. On code with a known flaky human test, Sonnet wrote a kept flaky test in 1 of 41 projects and Haiku in 2, against 4 for GPT 5.4 mini and 7 for GPT 4.1 mini (Table 8). In E1 the pooled LLM rate cannot be told apart from the human rate (Section 4.1). What differs is the cause. Most LLM tests share no state between tests, which removes the cause of both order dependent human flaky tests in E1 (Section 4.4), and each model has its own failure habit (Table 11). With a state restoring fixture (M2, Section 4.5) added after generation, kept flaky E2 tests drop from 49 to 10. |
| Why only one shot Haiku? | Haiku 4.5 was only the first model where we saw flips. E2 compares four models from two families under the same plain prompt. There Claude Haiku 5.5, a newer model than the Haiku 4.5 of E1, has 7 kept flaky tests in 1,908, all in two projects whose code is itself nondeterministic. Section 4.4 explains why Haiku 4.5 flipped in E1 where Sonnet did not. |
| 72 LLM suites does not equal 36 plus 24. | Our error. It was 60 LLM suites (36 on October 1, 24 on October 22) plus 12 human suites. With 24 OpenAI suites added, E1 now has 96 suites. |
| What are AG-S, OS-S, OS-H? | Every condition is now defined in Table 3 before it is used. |
| The next steps did not address the shortage of flaky tests. | E2 was the first step, and its extension batch is now done (Table 6). Section 7 adds 8 suites per model for every E2 project (about four times as many flaky suites at current rates) and an order dependent arm. |

**Table 3: Conditions. All generation is one shot with no test execution, except AG-S. In the codes AG is agent, OS one shot, TP TestPilot style, PL plain, S Sonnet, H Haiku, G5 GPT 5.4 mini, and G4 GPT 4.1 mini. E1 suites were generated earlier than E2, so some Claude versions differ.**

| Code | Model | Prompt |
|---|---|---|
| Human | library developers | none |
| AG-S | Claude Sonnet 5, agent that runs and fixes its tests | feature |
| OS-S | Claude Sonnet 5.5, one shot | feature |
| OS-H | Claude Haiku 4.5, one shot | feature |
| TP-S | Claude Sonnet 5.5, one shot | TestPilot style |
| PL-S | Claude Sonnet 5.5, one shot | plain |
| PL-G5 | GPT 5.4 mini (OpenAI), one shot | plain |
| PL-G4 | GPT 4.1 mini (OpenAI), one shot | plain |

**How suites were generated.** E2 uses Claude Sonnet 5.5, Claude Haiku 5.5, GPT 5.4 mini, and GPT 4.1 mini. Model versions were read from the generation logs. Claude suites were written by Claude Code subagents, which read the source files themselves and were told not to open any test file or run any command. An audit of all 164 Claude E2 transcripts (`results/generation_audit.json`) found that none opened a project test file or ran a test. Sonnet did use the shell to read source and write its file in 79 of 82 suites and Haiku in 6, which the prompt forbade, and in one Sonnet kevinarpe-rambutan3 suite a `grep` over the project listed the names of 10 test files, not the flaky one, without showing their contents. The E2 plain prompt also names the code to test, unlike the E1 plain prompt. OpenAI suites came from the API, with the target source files pasted into the prompt, so those models never saw a test file. No generated test reuses the name of the project's flaky human test. The two pipelines differ in more than the model, which Section 4.6 discusses.

**Flaky.** A test is flaky if it both passed and failed on the same machine. Every run starts a fresh pytest process. Runs shuffle the test order (with the pytest-randomly plugin) and change the hash seed. Some runs also change the time zone or add CPU and memory load with stress-ng, with 10 configurations sampled over CPU workers, CPU load, memory workers, and memory size, as Shaker [11] does, a tool that finds flaky tests by loading the machine. A generated test counts only if a one run generate and filter step would keep it, as in TestPilot, because that is what a developer would keep. A *test* is one pytest test case, so each parametrized case counts.

**Three parts.**

- **E1, 12 libraries.** Twelve Python libraries, each with its human suite and one suite per LLM condition (96 suites).
- **E2, matched study.** Projects where Gruber et al. found a flaky test. We reproduce that test, then ask each model to test the same code without showing it any test file.
- **E3, why.** Targeted reruns and a polluter search for every flaky generated test, a static comparison of how each condition builds its tests, and direct experiments.

We oversample stratum 1 on purpose, because nondeterminism enters through those modules. Our 12 libraries were chosen in the

**Table 4: Subject funnel for E1 (top 5,000 PyPI packages)**

| Rule | Left |
|---|---|
| Top 5,000 by downloads (last 30 days) | 5,000 |
| Is a library (no stubs, pytest plugins) | 4,701 |
| Links a GitHub repository | 4,131 |
| Ships a pure Python wheel | 3,296 |
| At most 3 required dependencies, runs on Python 3.12 | 2,188 |
| Repository has test files | 1,959 |
| Stratum 1, imports time, random, threading, asyncio, ... | 1,135 |
| Stratum 2, imports none of them | 824 |
| Pass rate of the last rule, suite passes offline in under 60 s, on 6 sampled packages per stratum | 5 of 12 |

pilot, before the funnel existed, and all 12 pass every rule (ten in stratum 1, and tinydb and tabulate in stratum 2 as controls).

# 4 RESULTS SINCE PROGRESS REPORT 1

## 4.1 E1, the large rerun campaign

All E1 suites ran on GitHub Actions Linux runners with 2 virtual CPUs. Every run shuffles the order and the hash seed. Half the runs do only that, a quarter also set a random time zone, and a quarter add stress-ng. Generated tests ran 600 times and human tests, whose suites are about five times larger, 200 times, so a flip is easier to catch in an LLM test. Table 5 has the results. In all, 6.3 million test executions ran.

**Table 5: E1 campaign. Generated tests are those the one run filter keeps. Human and AG-S rows count every test (AG-S already ran and fixed its own suite). Flaky % has a 95% Wilson interval.**

| Condition | Tests | Runs each | Flaky | Flaky % (95% CI) |
|---|---|---|---|---|
| Human | 5,587 | 200 | 5 | 0.09 (0.04 to 0.21) |
| AG-S | 1,511 | 600 | 1 | 0.07 (0.01 to 0.37) |
| OS-S | 1,406 | 600 | 0 | 0.00 (0.00 to 0.27) |
| OS-H | 1,595 | 600 | 3 | 0.19 (0.06 to 0.55) |
| TP-S | 913 | 600 | 0 | 0.00 (0.00 to 0.42) |
| PL-S | 1,191 | 600 | 0 | 0.00 (0.00 to 0.32) |
| PL-G5 | 852 | 600 | 1 | 0.12 (0.02 to 0.66) |
| PL-G4 | 900 | 600 | 10 | 1.11 (0.60 to 2.03) |

- **We cannot tell the LLM tests apart from the human tests, except PL-G4.** All LLM conditions together have 15 flaky tests in 8,368 (0.18%) against 5 in 5,587 human tests (0.09%, Fisher's exact test $p = 0.25$). PL-G4 alone differs (10 in 900, $p < 0.001$), and without it the LLM rate is 0.07% ($p = 0.75$). Generated tests ran three times as often, which favors finding LLM flips. Given each test's failure rate, 200 runs would have caught an expected 3.6 of the 5 non PL-G4 flips. This is no detectable difference, not proof of none, since without PL-G4 the upper bound allows an LLM rate about 1.8 times the human rate.
- **PL-G4 is the outlier, but from one cause.** Its 10 flaky tests are cases of three humanize test functions that depend on test order (Section 4.4). They are one bug repeated.
- **The other LLM flaky tests are rare.** Two tenacity tests (AG-S and PL-G5) compare floating point clock arithmetic at an exact value or boundary and failed 6 and 2 times in 600 runs. Three OS-H tests depend on the time.

Progress Report 1 counted 7 human flaky tests, but two (filelock) flipped only on our macOS laptop. The five here are two order dependent arrow locale tests and three pybreaker tests that flipped only under stress-ng.

## 4.2 E2, the matched study

**Reproducing known flaky tests.** Gruber et al. labeled by hand a sample of 100 flaky Python tests that are not order dependent. 42 are caused by the network, which leaves 58. Before any generation we fixed a rule for a second batch from their unlabeled data, the extension batch (`scripts/select_ext.py`), and we stopped when its candidate list ran out, not at a $p$ value. From each of the 226 other projects with such a flaky test, we dropped the project if its name or every flaky test name points to the network, and otherwise took one test, preferring those not marked as infrastructure and with a failure rate closest to one half. For every test we checked out the project at the recorded commit, installed it in a fresh Python 3.8 environment, and ran the test up to 150 times (Table 6).

**Table 6: Reproducing flaky tests of Gruber et al.**

| Outcome | Labeled | Extension |
|---|---|---|
| Candidates | 100 | 226 |
| Network (label, or name for extension) | 42 | 50 |
| Tried | 58 | 176 |
| Repository gone or install failed | 3 | 5 |
| Always failed (old dependencies, data files) | 21 | 107 |
| Never failed (up to 150 runs) | 13 | 34 |
| Reproduced (passed and failed) | 21 | 30 |
| Used for the matched study | 17 | 24 |

We left out 4 of the first 21 (two need downloads or a web API, and two tests of Hypothesis, a property based testing library, mostly hit the time limit), and 6 of the 30 that still depend on a network service or on hardware. The extension tests had no label, so Manas labeled each human test after it reproduced. Of the 41 projects, 35 are randomness, and one each is time, concurrency, async wait, and floating point, with two in other categories. So E2 is still mostly about randomness.

**Generating tests for the same code.** For each project we found the code the flaky test exercises (`scripts/matched_targets.py`, then checked by hand) and asked each model for 10 to 25 tests of it, two independent samples per model. Every suite then ran 200 times on the same runner, half in file order without pytest-randomly and half shuffled, with a new hash seed each run and no time zone change or stress-ng load. For the extension batch the runner also records which half each failure came from. The known flaky human test ran 200 times

next to it as a control and failed at least once in 38 of 41 projects (860 failures in 8,200 runs). Table 7 lists the three that did not. On the other 38 projects the Sonnet against GPT 4.1 mini split is 0 : 5 ($p = 0.063$).

**Table 7: Human control tests that did not fail in the E2 runs. Later counts the 200 E2 runs, plus 300 recheck runs for data-genie.**

| Project | Reproduction | Later | Likely reason |
|---|---|---|---|
| docci | 1 of 150 | 0 of 200 | rare failure |
| pyalcs | 1 of 150 | 0 of 200 | rare failure |
| data-genie | 98 of 150 | 0 of 500 | 300 ms Hypothesis deadline missed on a slow runner session (1 of 120 on a laptop) |

**Table 8: E2 by model, 41 projects, plain prompt, 200 runs per test. Suites with one means suites with a kept flaky test. A few suites did not import, so the suite counts differ.**

| Model | Suites | Kept | Flaky kept | Suites with one | Removed |
|---|---|---|---|---|---|
| Sonnet 5.5 | 81 | 1,883 | 1 | 1 | 1 |
| Haiku 5.5 | 81 | 1,908 | 7 | 4 | 0 |
| GPT 5.4 mini | 80 | 1,028 | 25 | 4 | 1 |
| GPT 4.1 mini | 80 | 1,070 | 16 | 10 | 12 |

**Table 9: Model comparisons on the clustered units, per suite (Fisher's exact test on suites with a kept flaky test) and per project (exact sign test on projects where only one of the two models has one; the counts are first model : second model)**

| Pair | Suite $p$ | Projects | Project $p$ |
|---|---|---|---|
| Sonnet vs GPT 4.1 mini, first 17 | 0.053 | 0 : 4 | 0.13 |
| Sonnet vs GPT 4.1 mini, extension 24 | 0.11 | 0 : 2 | 0.50 |
| Sonnet vs GPT 4.1 mini, all 41 | 0.005 | 0 : 6 | 0.031 |
| Haiku vs GPT 4.1 mini, all 41 | 0.10 | 1 : 6 | 0.13 |
| Sonnet vs Haiku, all 41 | 0.37 | 0 : 1 | 1.0 |
| Haiku vs GPT 5.4 mini, all 41 | 1.0 | 2 : 4 | 0.69 |
| Sonnet vs GPT 5.4 mini, all 41 | 0.21 | 1 : 4 | 0.38 |
| GPT 5.4 mini vs GPT 4.1 mini, all 41 | 0.16 | 3 : 6 | 0.51 |

"Removed" counts flaky tests a one run filter would drop, either because they failed in the first run or because they flip only after a test that did (2 GPT 4.1 mini tests, Section 4.3). Table 9 compares the models on suites and projects, because flaky tests cluster. Our main test is the project level sign test, which counts each project once. The suite level Fisher test treats a model's two suites for one project as independent, so we show it only for reference. With six model pairs the Bonferroni threshold is 0.0083, and no pair reaches it at the project level. Sonnet against GPT 4.1 mini comes closest (0 : 6 projects, $p = 0.031$; the suite level $p = 0.005$ ignores clustering). We chose the project level test after seeing these data, which is one reason step 1 of Section 7 is a held out sample with the test fixed in advance. For Sonnet against GPT 4.1 mini neither batch is significant alone, so we treat the gap as a lead to test with more suites per project, not as a finding.

## 4.3 Root causes

Manas labeled the root cause of every flaky LLM test with the categories of Luo et al. [6] and Gruber et al. [2]. Each label comes from the test, the code it calls, and a reproduction of the failure, and each was checked again against the reruns below. `results/root_causes/labels.json` holds 79 of the 91 labels (77 LLM test records, one covering 3 parametrized cases, plus one human test record) with the quoted evidence, and a reproduction for the 39 tests of the extension batch and guard v3. The 12 others, all E1, are in `results/why_analysis.md`. For every flaky E2 test except one removed guard v1 test in gamble, which we did not rerun, we also ran `scripts/od_diagnose.py`. It reruns the test alone 30 times, 10 of them with a fixed hash seed, and if the test never fails alone it runs each other test of the suite before it, as iDFlakies [4], iFixFlakies [10], and their Python counterpart iPFlakies [12] do. Table 10 has the counts.

**Table 10: Root causes of the 91 flaky LLM tests. "Rem." means removed by a one run filter.**

| | E1 | E2 plain | | E2 guard | |
|---|---|---|---|---|---|
| Root cause | kept | kept | rem. | kept | rem. |
| Randomness, unseeded generator | 0 | 12 | 9 | 4 | 2 |
| Randomness, salted `hash()` | 0 | 3 | 1 | 6 | 1 |
| Unstable object hash in the code | 0 | 6 | 1 | 0 | 0 |
| Test order dependency | 10 | 24 | 3 | 0 | 0 |
| Time | 3 | 4 | 0 | 0 | 0 |
| Floating point on a clock value | 2 | 0 | 0 | 0 | 0 |
| Total | 15 | 49 | 14 | 10 | 3 |

**Table 11: Kept flaky E2 tests (plain prompt) by model and root cause**

| Root cause | Sonnet | Haiku | GPT 5.4 | GPT 4.1 |
|---|---|---|---|---|
| Unseeded generator | 0 | 0 | 0 | 12 |
| Salted `hash()` | 0 | 3 | 0 | 0 |
| Unstable object hash | 1 | 4 | 0 | 1 |
| Test order dependency | 0 | 0 | 24 | 0 |
| Time | 0 | 0 | 1 | 3 |

**Inherited from the code.** In 24 of the 25 kept flaky E2 tests that are not order dependent, the test flips for the same reason as the project's human flaky test. The exception is a pelicun test that draws from the same unseeded generator through a different function and checks a sample mean. Most randomness cases call code that draws from an unseeded global generator (NumPy, `random`, or PyTorch) and assert one outcome. In pelicun GPT 4.1 mini seeded NumPy at module level in one suite and inside earlier tests in the other, but pytest-randomly reseeds NumPy before every test in the shuffled runs, so those seeds had no effect there. Its 2 kept

tests fail only in shuffled runs, so a developer running plain pytest would not see them flip. The two hash causes are different. In cuckoopy the filter places items by Python's built in `hash()` of a string, which changes with `PYTHONHASHSEED` in every process, so seeding cannot help. In kevinarpe-rambutan3 the code's own `__hash__` returns the hash of a temporary `super` object, so it changes from call to call, and an `OrderedDict` that stores the object then raises `KeyError`. Tests that hash the matcher more than once, directly or by passing `None` with the library's error collector, can hit this bug, as the human test does. Here and in cuckoopy a flip exposes real nondeterminism in the code, so such a test arguably found a bug rather than being badly written. We still count it, because a developer sees it flip. The docci tests save the same workbook twice and compare the bytes, which differ when the two saves fall on either side of an even second, because each zip entry stores its save time with two second resolution.

**Introduced by the test.** All 24 order dependent kept tests come from three GPT 5.4 mini suites, and none of them failed in a file order run. All are *victims* in the iDFlakies sense: they pass alone and fail after another test, the *polluter*. Each suite has a polluter that changes how modules are loaded and does not undo it. In torch-gpt-2, four tests monkeypatch a submodule attribute and then call `importlib.reload` on the package, which copies the fake into the package. The monkeypatch undoes only the submodule, so 7 later tests see the fake. In jmcm, two tests remove jmcm from `sys.modules` and import it again while `builtins.__import__` is patched to fail for NumPy or `math`. If one runs first, pandas, SciPy, and patsy are first imported under the patch and stay half initialized, so 16 later tests fail. In pypfm, one test pops the package from `sys.modules` and imports a new copy, and a later test reloads the old copy. The polluter search agrees. Every one of the 24 passes alone in 30 runs, and for 23 of them a single earlier test makes it fail. The 24th, a jmcm test, fails after the NumPy polluter only when pytest-randomly has already imported NumPy, which our search runs without. The two GPT 4.1 mini victims in pylabeledrf follow an older pattern. A rejected polluter replaces a module level function with a fake and fails before restoring it.

**E1 timing tests.** The AG-S and PL-G5 tenacity tests build a fake timestamp from a real `time.monotonic()` reading and then check an exact value (PL-G5, `== 1.0`) or an exact boundary (AG-S, `>= 5` at an elapsed time of 5.0), which fails for some clock values. The two OS-H schedule tests build a clock time from the real clock, which wraps past midnight when the run starts late in the local day, so both failed only in the time zone mode. The OS-H tenacity test asserts a minimum elapsed time after two random waits whose sum can be shorter, so it fails in 21 to 24% of runs in every mode.

## 4.4 E3, why most LLM tests do not flip

`scripts/characterize.py` scans every test function in E1 for shared state, isolation helpers, clock and sleep use, timing bounds, and randomness. Table 12 shows the features most tied to order dependence for the plain prompt conditions. All conditions are in `results/why_analysis.md`.

**Why most LLM tests do not flip.** Human suites share setup through class and module scoped fixtures (19.5% of human test functions), and 4.2% have setup code with no matching teardown.

**Table 12: Share (%) of E1 test functions with each feature (functions in the source, before the filter, so parametrized cases count once)**

| Feature | Human | PL-S | PL-G5 | PL-G4 |
|---|---|---|---|---|
| Test functions | 3,077 | 927 | 897 | 985 |
| Any shared state between tests | 23.9 | 1.0 | 6.5 | 1.2 |
| Class or module scoped fixture | 19.5 | 0.0 | 0.0 | 0.0 |
| Touches a private attribute | 12.3 | 0.9 | 14.2 | 15.5 |
| Resets library state after the test | 0.3 | 8.6 | 9.8 | 0.0 |
| Builds a fresh library object | 18.5 | 48.1 | 34.0 | 46.2 |
| Setup without teardown | 4.2 | 0.0 | 0.0 | 0.0 |

The order dependent human flaky tests sit on that kind of shared state, a class scoped fixture in the two arrow tests. No LLM condition uses a fixture wider than one test or has setup without teardown, and LLM tests build a fresh library object two to two and a half times as often as human tests. LLM suites also cover less code (67 to 83% of lines against 94%), which may explain part of the E1 result. In E2 every model tests the same code that the flaky human test exercises.

**Why GPT 4.1 mini flips in E1.** The GPT models touch private attributes as often as human tests do (Table 12), and this is one example. Three PL-G4 humanize test functions (10 cases) call `monkeypatch.setattr(i18n._CURRENT, "locale", ...)`. `_CURRENT` is a `threading.local` whose `locale` attribute exists only after some earlier test has called `activate` or `deactivate`, so these tests pass only when such a test ran earlier. They are *brittles* in the iDFlakies sense, which fail alone and need a state setter to run first. The one run filter ran the file in its written order, where they pass. PL-S resets this state around every humanize test, and PL-G4 never does. With one added fixture that calls `deactivate()` around every test, none of the 10 flipped in 50 shuffled runs, against all 10 without it (`scripts/humanize_reset_check.py`).

**Table 13: E2 suites (82 per model, plain prompt, counting the few that did not import) with each habit, from `scripts/matched_habits.py`**

| Habit | Sonnet | Haiku | GPT 5.4 | GPT 4.1 |
|---|---|---|---|---|
| Seeds a random generator | 33 | 32 | 5 | 8 |
| Uses `monkeypatch` or `mock.patch` | 21 | 27 | 44 | 30 |
| Reloads a module or touches `sys.modules` | 0 | 0 | 13 | 0 |

**Why GPT 4.1 mini flips in E2.** The code under test draws random numbers itself, and Table 13 shows that the GPT models rarely seed. GPT 5.4 mini still wrote no unseeded randomness test. It uses `monkeypatch` most often, which fits replacing the random draws with fakes, though we did not count that directly. Sonnet and Haiku seed in about 40% of suites and often pick inputs that avoid the random path, as with `squeeze=True` in spreading_dye_sampler. GPT 4.1 mini does neither in most suites.

**Why GPT 5.4 mini flips in E2.** Its suites test the package itself, not only its functions. 13 of its 82 suites reload a module or read

or edit `sys.modules`, and no suite of the other three models does. Three of those 13 have a kept flaky test and one (limix-lmm) has a removed one. None of them restores the module state afterwards, and the file order filter run hides the problem.

**Why Haiku 4.5 flipped in Progress Report 1.** With the same prompt as one shot Sonnet, it wrote about twice as many tests, never froze the clock or faked a sleep where Sonnet did, and left less headroom in timing bounds (`results/why_analysis.md`).

## 4.5 Mitigation

We tried three kinds of mitigation, research question 4 of the proposal, on the E2 tests.

**Fixtures added after generation.** A fixture is a `conftest.py` added to each generated suite, with the tests unchanged. *M1* (`mitigations/seed_conftest.py`) seeds `random`, NumPy, and PyTorch inside each test call. It has to run there, after pytest-randomly reseeds, which is also why the module level seeding GPT 4.1 mini wrote in pelicun had no effect. *M2* (`mitigations/isolate_conftest.py`) does the same, imports the project's packages once before any test runs, and snapshots `sys.modules`, the globals of every loaded module, `sys.path`, the environment, and the working directory before each test, then restores whatever changed after teardown. The up front import is there for jmcm. Without it, a restore cannot help, because the polluter is the first to import pandas and leaves it half built (in a local check of 40 shuffled runs, a restore alone did not help, and adding the up front import removed every failure). Every suite reran 200 times as before (Table 14).

**Table 14: Kept flaky E2 tests with each fixture, 41 projects, kept tests in parentheses**

| Model | Plain | M1, seed | M2, seed and restore |
|---|---|---|---|
| Claude Sonnet 5.5 | 1 (1,883) | 2 (1,884) | 2 (1,884) |
| Claude Haiku 5.5 | 7 (1,908) | 5 (1,906) | 5 (1,906) |
| GPT 5.4 mini | 25 (1,028) | 25 (1,028) | 1 (1,027) |
| GPT 4.1 mini | 16 (1,070) | 3 (1,075) | 2 (1,073) |

We claim only the two changes of whole causes. The other changes are within detection noise, since a test that fails in 0.5% of runs shows up in 200 runs only about 63% of the time.

- **M1** removed all 12 unseeded generator tests from the kept flaky set. Eleven became stable, and one pelicun test now fails in every run, so the filter drops it.
- **M2** also removed all 24 order dependent tests (jmcm, torch-gpt-2, and pypfm), so GPT 5.4 mini goes from 25 to 1.
- **Left under both.** The clock (docci), the salted `hash()` (cuckoopy), and the code's own unstable hash (kevinarpe-rambutan3, whose failures depend on memory layout). These need a change to the test or to the code. One of the rare docci tests flipped only under the fixtures, and we count it there.
- **Cost.** Kept counts moved by at most 5 per model, but on the runners M2 made 3 stable tests fail in every run (2 in keplertools, 1 in limix-lmm). They pass with M2 on a laptop, and we have not yet found why.

Both fixtures and guard v3 were built on these same tests (M1 and M2 as of commit `7c05b6b`), so these are tuning results. The held out check is step 1 of Section 7, plus E1.

**Prompts.** We regenerated the 34 GPT 4.1 mini suites of the first 17 projects with three versions of a guard prompt (all three in `prompts/README.md`). Version 1 lists the usual causes of flakiness. Version 2 adds three concrete rules (seed inside the test body, do not assert on `hash()` of strings, use `monkeypatch` for module attributes). Version 3 limits seeding to the generators the code under test already uses, because version 2 made GPT 4.1 mini import PyTorch or NumPy into projects that do not have them.

**Table 15: GPT 4.1 mini guard prompts, first 17 projects (34 suites)**

| Prompt | Did not import | Kept | Flaky kept |
|---|---|---|---|
| Plain | 0 | 558 | 10 |
| Guard v1 | 0 | 550 | 9 |
| Guard v2 | 9 | 500 | 0 |
| Guard v3 | 5 | 429 | 1 |

The concrete rules work, but they cost suites. Guard v3 cut kept flaky tests from 10 to 1, but 5 of 34 suites did not import and the filter rejected a third of the tests, so 129 fewer tests survive than with the plain prompt. The one remaining test is a cuckoopy `hash()` case. The guard prompts also led GPT 4.1 mini to write more cuckoopy tests that hit `hash()` (6 kept in v1 and v3, none with the plain prompt), because they ask for tests of collisions and deletes. M1 removed the same 10 with no loss, so for unseeded randomness a fixture is the better tool.

**Longer rerun filters.** From the measured failure rate of each of the 74 kept flaky LLM tests, we estimated what $k$ extra filter runs would catch, separately for runs in file order and in shuffled order (`scripts/krun_estimate.py`, Table 16). It assumes each run fails independently at the measured rate. File order rates are measured for the 37 tests of the extension batch and guard v3, and for the others we use the rate over all runs (0 for the 10 PL-G4 humanize brittles), which overstates file order catches for tests that fail in one mode only. The 74 were also found by shuffled reruns, which favors shuffling. With that, 40 of the 74 never fail in file order, so a filter that only repeats the file order catches about a quarter of them even with 10 runs. Shuffling the extra runs nearly triples that. The 74 tests fail in a median 9.8% of their runs (0.2% to 50%), and 21 fail in fewer than 5% of all their runs (`fail_under_5pct_all_runs` in `krun_estimate.json`).

**Table 16: Expected share (%) of the 74 kept flaky LLM tests that $k$ extra filter runs would catch**

| Extra runs $k$ | 1 | 3 | 5 | 10 |
|---|---|---|---|---|
| File order | 7 | 16 | 20 | 26 |
| Shuffled order | 22 | 47 | 60 | 74 |

### 4.6 Threats to validity

- **Pipeline, not only model.** Claude suites came from Claude Code subagents that read files, and OpenAI suites came from the API with sources pasted in. The Claude prompt also adds that the tests are for a research experiment. Sampling settings and system prompts differ too, so the model comparison is a comparison of two pipelines with four models.
- **Scope of E2.** 35 of 41 projects are randomness projects. Each of time, concurrency, async wait, and floating point has one project, and E2 runs add no load, so finding no flaky test in the concurrency and async wait projects says little. The four time tests are all in docci.
- **Already noted above.** No model difference survives correction (Section 4.2), E1 gave LLM tests three times the runs, and 2 pelicun tests flip only because pytest-randomly reseeds NumPy.
- **What M2 restores.** M2 undoes changes to module state that a test may make on purpose. It removed flips in our suites, but it could also hide a real bug that leaks state, and it broke 3 stable tests on the runners, so we see it as a filter aid, not a fix to ship.
- **Old environments and few samples.** E2 runs on Python 3.8 with old pinned dependencies, and each model wrote only two suites per project.
- **Generation and labeling checks.** The transcript audit rules out reading the human tests, but the Claude subagents chose which source files to read, and the OpenAI models saw only the files we pasted. Whoever labeled a test could see which model wrote it, and each label has one labeler, so we have no agreement statistic yet (step 5 of Section 7).

## 5 MILESTONE GOALS AND STATUS

Table 17 lists the goals the proposal set for this milestone (its Intermediate Steps 2) and the next steps we promised in Progress Report 1.

## 6 CHALLENGES

- **Old projects are hard to run.** Only 51 of 234 known flaky tests reproduced. Most of the rest always failed because of old dependencies or missing data files.
- **Harness bugs.** In fuzzylogic (a `src` folder added to the path by hand) and fossor (pytest told to collect only `test/`) every generated suite first collected nothing. We fixed both and reran them.
- **Models invent names.** PL-G4 lost two whole E1 suites to imports of names the packages do not export, and in pymemorydb three models imported the package by its project name, which is not its import name.
- **A one run filter in file order hides order dependence.** That is how all 34 order dependent kept tests (24 victims and 10 brittles) got through.

## 7 NEXT STEPS AND RECOVERY PLAN

The steps are in priority order. Progress Report 3 is due November 21.

(1) **A held out sample in one pipeline (Shaunak, by November 13).** Generate 8 new plain suites per model for each of the 41 E2 projects, calling the Claude models through `claude -p` with no tools and the same pasted prompt as the OpenAI models, so all four models share one pipeline. These 1,312 suites are a fresh sample that we analyze alone, with the analysis fixed now: the per project sign test of `scripts/matched_stats.py` at commit `7c05b6b`, Bonferroni over six pairs ($p < 0.0083$), which a $0:8$ project split would pass ($p = 0.0078$). Every new suite runs 200 times plain, with M1, and with M2 as frozen, which tests the fixtures on tests they were not built for. From the M2 run (16.6 runner hours for 328 suites) this is about 200 runner hours, about half a day at the 20 parallel jobs GitHub gives a public repository. If it runs late we cut to 4 new suites per model, where the same $0:8$ project split would still pass.

(2) **Mitigation for every model and E1 (Shaunak, by November 18).** Run M1 and M2 on the E1 suites, run guard v3 for the other three models on E2, measure a 3 and 5 run shuffled filter directly instead of estimating it, find why M2 broke 3 tests on the runners, and compare M2 with an iFixFlakies style fix that runs a cleaner from the same suite. For each we report flaky tests kept, good tests lost, coverage, the failure rate of the flaky tests left, the scanner's risk rate, and cost.

(3) **An order dependent arm (Manas, by November 20).** Gruber et al. [2] also found 4,461 order dependent tests in 615 projects. For 10 of these projects (5 if time runs short) we will generate tests for the same code and run them inside the human suite in shuffled orders. GPT 5.4 mini's reload habit makes this sharper, because its tests could become polluters of human tests.

(4) **Non randomness projects (Shaunak and Manas, by November 20).** E2 has only six projects whose flaky test is not about randomness. The extension rule took one test per project, so we will try 40 more time, concurrency, and async wait tests from the same unlabeled data, either other tests of those projects or tests that always failed only because of old dependencies, retried with pinned versions. At our 22% yield that is about 9 projects. Their E2 runs, and reruns of the six existing ones, add the stress-ng mode.

(5) **Root causes (Manas, ongoing).** Label every new flaky test the same way as above. By November 16 Shaunak will label the existing 91 again without seeing the model or Manas's label, and we will report Cohen's kappa.

(6) **Delay injection (if time allows).** FlakeRake [8] on the E1 tests with timing bounds.

**Check point and fallback.** On November 13 we check steps 1, 3, and 4. Our reproduction yield so far is 51 of 234 (22%), so step 3 will try about 45 order dependent tests to get 10 projects. If fewer than 5 have reproduced by then, we stop at the ones that did and move that time to step 4, and if fewer than 3 step 4 tests have reproduced

**Table 17: Goals for this milestone**

| Goal | Status | Notes |
|---|---|---|
| Rerun every flagged test at least 1,000 times on the GitHub runners, with and without CPU stress | Partly | For Progress Report 1, the flagged OS-H, OS-S, and AG-S tests got 1,500 runs, but the rest fewer. E1 now runs every generated test 600 times and every human test 200 times. |
| Add a non Claude model | Done | GPT 5.4 mini and GPT 4.1 mini, plain prompt, in E1 and E2 (not the TestPilot style prompt). |
| Add the plain prompt | Done | Added October 22 (PL-S), now also used for every E2 model. |
| Measure scanner precision and recall | Done | Recall 10 of 96 Gruber et al. flaky tests (most nondeterminism is inside the library), and it flags 35 of 714 never flaky tests. |
| Measure coverage of every suite | Done | Mean line coverage 94.4% for human suites, 67.0% (PL-G4) to 82.6% (PL-S) for LLM conditions, without filelock (`results/coverage/`). |
| Search for order dependent tests in the style of iDFlakies [4], a tool that reruns tests in many orders to find order dependence | Done for E2 | Reruns shuffle, and `od_diagnose.py` finds polluters. Running generated tests inside the human suite (Progress Report 1) found three AG-S victims, and step 3 of Section 7 extends that to new projects with known order dependent tests. |
| Add four new subjects from the funnel | Replaced | E2 adds 41 projects with known flaky code, which attacks the shortage of flaky tests more directly. |
| Pick stress-ng configurations with Shaker's selection step (keep the configurations that expose the most known flaky tests) | Not done | Only three human tests flip only under stress-ng, too few to choose configurations from. |
| Run FlaPy [1] (the rerun tool of Gruber et al.) on every suite | Dropped | On two suites (1,996 tests) FlaPy flagged the 3 flaky tests our E1 runs found, and, as Progress Report 1 said, missed a deadline test that failed once under stress. Our harness also reruns in fresh processes, in file and shuffled order. |
| Start the mitigation early | Done | Two fixtures, three prompt versions, and the rerun filter estimate (Section 4.5). |

we add their stress-ng runs to the E1 libraries instead. Step 2 is checked on November 18, and if M2 still breaks stable tests we report M1 only. Step 6 is the first to cut. For the final report we aim for at least 40 suites with a kept flaky test from at least three models, and at least 8 E2 projects outside randomness.

**Division of work.** Since Progress Report 1, Manas did the literature review and the root cause labeling of the new flaky tests, including the human tests of the extension batch. Shaunak built the reproduction and matched study pipeline, generated the OpenAI suites, ran the E1 campaign and the mitigation trials, and wrote the analysis scripts and this report. Manas will also write the related work section of the final report.

## 8 RELATED WORK

Luo et al. [6] give the root cause categories we use, and Gruber et al. [2] show what flakiness looks like in Python and provide the tests behind E2. TestPilot [9] is the generate and filter pipeline we imitate, and CodaMosa [5] uses an LLM to help a search based test generator. Both keep a test after one passing run, which our estimate in Section 4.5 suggests is not enough for flaky tests. Shaker [11] gives our stress-ng configurations, and iDFlakies [4], iFixFlakies [10], and iPFlakies [12] the polluter search and the victim and brittle terms. Parry et al. [7] survey the field. The closest study is Gruber et al. [3], who measured flakiness in tests from search based generators such as EvoSuite and Pynguin. We ask the same question for LLM generators, compare against the human tests of the same code, and test mitigations in the generate and filter step.

## 9 CODE, DATA, AND DOCUMENTATION

Everything is in the public repository https://github.com/shaunak1206/ece382v-flaky-llm-tests. `README.md` gives the setup and the command for every experiment, including the scripts new since Progress Report 1, `docs/DATA.md` documents every result file and field, `prompts/README.md` has every prompt, and `results/root_causes/` has the labels.

## REFERENCES


[1] Martin Gruber and Gordon Fraser. 2023. FlaPy: Mining Flaky Python Tests at Scale. In *ICSE Companion*.

[2] Martin Gruber, Stephan Lukasczyk, Florian Kroiß, and Gordon Fraser. 2021. An Empirical Study of Flaky Tests in Python. In *ICST*. 148–158.

[3] Martin Gruber, Muhammad Firhard Roslan, Owain Parry, Fabian Scharnböck, Phil McMinn, and Gordon Fraser. 2024. Do Automatic Test Generation Tools Generate Flaky Tests?. In *ICSE*.

[4] Wing Lam, Reed Oei, August Shi, Darko Marinov, and Tao Xie. 2019. iDFlakies: A Framework for Detecting and Partially Classifying Flaky Tests. In *ICST*. 312–322.

[5] Caroline Lemieux, Jeevana Priya Inala, Shuvendu K. Lahiri, and Siddhartha Sen. 2023. CodaMosa: Escaping Coverage Plateaus in Test Generation with Pre-trained Large Language Models. In *ICSE*.

[6] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *FSE*. 643–653.

[7] Owain Parry, Gregory M. Kapfhammer, Michael Hilton, and Phil McMinn. 2021. A Survey of Flaky Tests. *ACM TOSEM* 31, 1 (2021).

[8] Shanto Rahman, Aaron Massey, Wing Lam, August Shi, and Jonathan Bell. 2024. Automatically Reproducing Timing-Dependent Flaky-Test Failures. In *ICST*.

[9] Max Schäfer, Sarah Nadi, Aryaz Eghbali, and Frank Tip. 2024. An Empirical Evaluation of Using Large Language Models for Automated Unit Test Generation. *IEEE TSE* 50, 1 (2024), 85–105.

[10] August Shi, Wing Lam, Reed Oei, Tao Xie, and Darko Marinov. 2019. iFixFlakies: A Framework for Automatically Fixing Order-Dependent Flaky Tests. In *ESEC/FSE*. 545–555.

[11] Denini Silva, Leopoldo Teixeira, and Marcelo d'Amorim. 2020. Shake It! Detecting Flaky Tests Caused by Concurrency with Shaker. In *ICSME*.

[12] Ruixin Wang, Yang Chen, and Wing Lam. 2022. iPFlakies: A Framework for Detecting and Fixing Python Order-Dependent Flaky Tests. In *ICSE Companion*. 120–124.